Часть 1. Основы Python • Тема 1.5

Циклы. Занятие 2

К концу занятия ты сможешь:

- рисовать таблицы и фигуры из символов циклом внутри цикла;
- получать случайные числа и случайные символы;
- узнавать, что цикл прошёл до конца без break;
- писать программу, которая работает, пока её не остановят.

Разминка

1. Сколько чисел выведет цикл for i in range(1, 10) и какое из них последнее?

2. Чем break отличается от continue?

3. Что выведет программа справа? Сначала подумай, потом проверим.

main.py

```
for number in range(1, 8):
    if number == 3:
        continue  # пропускаем 3
    if number == 6:
        break  # на 6 выходим
    print(number)
```

Вот что у тебя получится сегодня

Игра «Дуэль кубиков». Пять раундов, в каждом ты и компьютер бросаете по 3 кубика. У кого сумма больше, тот выиграл раунд.

Чтобы её написать, не хватает трёх вещей:

- цикла внутри цикла;
- случайных чисел;
- цикла, который крутится, пока не скажешь «нет».

Вывод в консоли

```
Раунд 1. Ты: 4 4 3 = 11, компьютер: 9
Раунд 2. Ты: 1 1 3 = 5, компьютер: 10
Раунд 3. Ты: 1 4 5 = 10, компьютер: 11
Раунд 4. Ты: 2 5 3 = 10, компьютер: 11
Раунд 5. Ты: 1 3 3 = 7, компьютер: 9
Счёт 1:4
Победил компьютер
Сыграем ещё? (да/нет): нет   <- ввёл пользователь
Спасибо за игру
```

Стрелка — пометка для тебя, в программе её нет. Числа случайные: у тебя будут другие.

План занятия

УРОК 1

Вложенные циклы

Цикл внутри цикла: таблицы, поля и фигуры из символов.

Практика: таблица умножения

УРОК 2

Случайные числа

random.randint и random.choice: кубики, буквы, уровни.

Практика: генератор паролей

УРОК 3

else у цикла и while True

Практика: сейф с PIN-кодом.

Мини-проект: «Дуэль кубиков»

Вспомним: for, range, break, continue

Запись	Что делает
for letter in "мяч":	перебирает символы строки по очереди: м, я, ч
range(1, 5)	даёт числа 1, 2, 3, 4 — конец не входит
range(0, 10, 2)	шаг 2: 0, 2, 4, 6, 8
break	сразу заканчивает весь цикл
continue	пропускает остаток шага, цикл идёт дальше
count += 1	счётчик: сколько раз что-то случилось

Урок 1 • 35 минут

Блок 1 из 3. Вложенные циклы

В этом блоке ты научишься повторять повторение: рисовать таблицы, игровые поля и фигуры из символов.

Что такое вложенный цикл

Вложенный цикл — цикл, который стоит внутри другого цикла.

Как школьная неделя: **внешний цикл** перебирает дни, а **внутренний** — уроки в каждом дне.

Сначала проходят все уроки понедельника, потом все уроки вторника, и так до пятницы.

Внешний цикл: для каждого дня

Внутренний цикл: для каждого урока

провести урок

день закончился, следующий день

Зачем: нарисуем прямоугольник 3 × 5

Было: 9 строк, одно и то же три раза

main.py

```
for j in range(5):
    print("*", end="")
print()
for j in range(5):
    print("*", end="")
print()
for j in range(5):
    print("*", end="")
print()
```

Стало: 4 строки, цикл в цикле

main.py

```
for i in range(3):
    for j in range(5):
        print("*", end="")
    print()
```

Вывод в консоли (у обеих программ)

```
*****
*****
*****
```

Нужно 30 строк? Слева — 90 строк кода, справа — меняем одно число: range(30).

Как записывается

Шаблон

```
for строка in range(...):           # внешний цикл
    for столбец in range(...):    # внутренний: отступ 4
        print(..., end=" ")       # отступ 8: много раз
    print()                       # отступ 4: раз на строку
```

end=" "
печатает без перехода на новую строку (тема 1.2)

print()
без аргументов — только переход на новую строку

Отступ
решает, к какому циклу относится строка

Пример: места в кинозале

main.py

```
# Места в кинозале: 3 ряда по 4 места
for row in range(1, 4):
    for seat in range(1, 5):
        print(f"{row}-{seat}", end=" ")
    print()  # переход на новую строку
```

Вывод в консоли

```
1-1 1-2 1-3 1-4
2-1 2-2 2-3 2-4
3-1 3-2 3-3 3-4
```

Разбор

row — номер ряда, внешний цикл: 1, 2, 3.

seat — номер места, внутренний цикл: 1, 2, 3, 4.

Внутренний цикл печатает весь ряд в одну строку.

print() после него закрывает ряд.

По шагам: кто за кем

main.py

```
for row in range(1, 3):
    for seat in range(1, 4):
        print(f"{row}-{seat}", end=" ")
    print()
```

Вывод в консоли

```
1-1 1-2 1-3
2-1 2-2 2-3
```

Внутренний цикл проходит целиком на каждом шаге внешнего: 2 × 3 = 6 печатей.

Шаг	row	seat	Что произошло
1	1	1	печать 1-1
2	1	2	печать 1-2
3	1	3	печать 1-3
4	1	—	print(): новая строка
5	2	1	печать 2-1
6	2	2	печать 2-2
7	2	3	печать 2-3
8	2	—	print(): новая строка

Живой кодинг: таблица умножения

1. Внешний цикл: i от 1 до 4 — номер строки.
2. Внутренний цикл: j от 1 до 4 — номер столбца.
3. Внутри печатаем i * j с end=" ".
4. Запускаем и смотрим на вывод: всё ли как ожидали?
5. Если нет — ищем причину по выводу, а не наугад.
6. Переход на новую строку после каждой строки таблицы.

Что должно получиться

Вывод в консоли

```
1 2 3 4
2 4 6 8
3 6 9 12
4 8 12 16
```

Внутренний цикл зависит от внешнего

main.py

```
for i in range(1, 5):
    for j in range(i):  # граница — переменная i
        print("#", end="")
    print()
```

Вывод в консоли

```
#
##
###
####
```

i	range(i)	решёток
1	0	1
2	0, 1	2
3	0, 1, 2	3
4	0, 1, 2, 3	4

Чем дальше строка, тем длиннее внутренний цикл. Так рисуют лесенки и треугольники.

Ловушка: лишние 4 пробела ломают картинку

Сломано: print() внутри внутреннего цикла

main.py

```
for row in range(1, 3):
    for seat in range(1, 4):
        print(f"{row}-{seat}", end=" ")
        print()  # отступ 8 пробелов
```

Вывод в консоли

```
1-1
1-2
1-3
2-1
2-2
2-3
```

Правильно: print() после внутреннего цикла

main.py

```
for row in range(1, 3):
    for seat in range(1, 4):
        print(f"{row}-{seat}", end=" ")
    print()  # отступ 4 пробела
```

Вывод в консоли

```
1-1 1-2 1-3
2-1 2-2 2-3
```

Ошибки Python нет — программа просто делает не то. Ищи, какая строка выполняется слишком часто.

Ловушка: нет отступа — IndentationError

main.py

```
for row in range(1, 3):
    for seat in range(1, 4):
    print(f"{row}-{seat}", end=" ")
    print()
```

Вывод в консоли

```
  File "/tmp/run/ex/indent_error/main.py", line 3
    print(f"{row}-{seat}", end=" ")
    ^^^^^
IndentationError: expected an indented block after 'for' statement on
line 2
```

traceback — сообщение об ошибке. Читай снизу вверх: внизу тип и причина — «после for на строке 2 ждали блок с отступом». Выше — строка 3, где Python споткнулся.

Правило: что с отступом под внутренним циклом — повторяется на каждом его шаге; что с отступом 4 после него — один раз на шаг внешнего.

Проверь себя

1. Внешний цикл делает 5 шагов, внутренний — 3. Сколько раз выполнится строка внутри внутреннего цикла?

2. Что будет с выводом кинозала, если убрать print() после внутреннего цикла?

3. В треугольнике range(i). Почему первая строка — одна решётка, а не ноль?

4. Что будет, если в кинозале поменять местами строки двух for (seat снаружи, row внутри)?

Задание: «Таблица умножения»

Шпаргалка для младшего брата или сестры.

- Программа спрашивает: «Размер таблицы: ».
- Выводит таблицу умножения от 1 до введённого числа: строк и столбцов столько, сколько ввели.
- Числа стоят ровными столбцами.

Подсказка. "\t" — символ табуляции, прыжок к следующему столбцу, как клавиша Tab: print(число, end="\t"). Не забудь: range(1, size) не включает size.

★ **Успел раньше?** Выведи над таблицей строку с номерами столбцов и строку из чёрточек под ней.

Застрял — 15 минут ищешь сам, потом спрашиваешь: что делал, что ожидал, что получил.

Что должно получиться

```
Вывод в консоли
Размер таблицы: 5   <- ввёл пользователь
1       2       3       4       5
2       4       6       8       10
3       6       9       12      15
4       8       12      16      20
5       10      15      20      25
```

Стрелка — пометка для тебя, в программе её нет.

Проверь себя

- Ввёл 5 — ровно 5 строк по 5 чисел.
- Последнее число — 25, то есть 5 × 5.
- Двузначные числа не сдвигают столбцы.

★★ «Шахматная доска»

Нарисуй доску 8 × 8, где # — тёмная клетка, а точка — светлая. Как на настоящей доске, цвета чередуются и по строкам, и по столбцам.

Подумай сам: как по номеру строки и номеру столбца понять, какого цвета клетка? Попробуй выписать номера нескольких тёмных клеток.

Размер доски задай одной константой SIZE — тогда доска 10 × 10 получится заменой одного числа.

Вывод в консоли

```
#.#.#.#.
.#.#.#.#
#.#.#.#.
.#.#.#.#
#.#.#.#.
.#.#.#.#
#.#.#.#.
.#.#.#.#
```

Урок 2 • 40 минут

Блок 2 из 3. Случайные числа

В этом блоке ты научишься бросать кубик, выбирать случайную букву и делать программы, которые каждый раз работают по-разному.

Модуль random: случайность в программе

Модуль random — готовый набор функций для случайных чисел. Как коробка с игральными кубиками: открыл — и бросай.

Зачем. Без случайности игра каждый раз одинаковая: если в «Угадай число» всегда загадано 7, второй раз играть уже неинтересно.

Пока просто запомни эту запись

```
import random
```

Первая строка программы. Она подключает модуль, как import math в теме 1.3. Подробно — в теме 3.4.

random.randint(a, b): бросаем кубик

random.randint(a, b) — случайное целое число от a до b. **Обе границы входят**: randint(1, 6) может дать и 1, и 6.

main.py

```
import random

dice = random.randint(1, 6)  # от 1 до 6 включительно
print(f"Выпало: {dice}")
```

Запуск 1

```
Выпало: 2
```

Запуск 2

```
Выпало: 2
```

Запуск 3

```
Выпало: 4
```

У тебя числа будут другие. Одно и то же число может выпасть два раза подряд — как у настоящего кубика.

random.choice(строка): случайный символ

random.choice(строка) — один случайный символ из строки. Как вытянуть карточку из колоды не глядя.

main.py

```
import random

letters = "АБВГДЕЖЗИКЛМНОПРСТ"
letter = random.choice(letters)  # одна случайная буква
print(f"Назови город на букву {letter}")
```

Запуск 1

```
Назови город на букву К
```

Запуск 2

```
Назови город на букву Т
```

Строку символов задаём сами: можно "0123456789" или "+-*".

Символ, который встречается в строке чаще, и выпадает чаще.

Ловушка: randint и range считают по-разному

main.py

```
import random

for number in range(1, 6):
    print(number, end=" ")
print()
for throw in range(20):
    print(random.randint(1, 6), end=" ")
print()
```

Вывод в консоли

```
1 2 3 4 5
5 5 2 6 5 4 3 4 2 2 1 1 2 6 6 5 2 1 5 6
```

Запись	Числа от … до …
range(1, 6)	1 … 5 — без 6
random.randint(1, 6)	1 … 6 — с 6

Хочешь кубик — пиши randint(1, 6), а не randint(1, 7).

Ловушка: забыли import random

main.py

```
dice = random.randint(1, 6)
print(dice)
```

Вывод в консоли

```
Traceback (most recent call last):
  File "/tmp/run/ex/nameerror/main.py", line 1, in <module>
    dice = random.randint(1, 6)
           ^^^^^^
NameError: name 'random' is not defined. Did you forget to import 'random'?
```

Читаем traceback снизу вверх

1. Последняя строка: NameError — имя random неизвестно. Python даже подсказывает: «забыл import random?»
2. Выше: строка 1, значки ^^^^^^ под словом random.
3. Исправление: первой строкой файла пишем import random.

Случайное число в цикле

main.py

```
import random

for throw in range(1, 6):
    dice = random.randint(1, 6)
    print(f"Бросок {throw}: {dice}")
```

Вывод в консоли

```
Бросок 1: 4
Бросок 2: 4
Бросок 3: 2
Бросок 4: 5
Бросок 5: 6
```

throw	dice	Что было
1	4	новый бросок
2	4	новый бросок
3	2	новый бросок
4	5	новый бросок
5	6	новый бросок

randint стоит внутри цикла, поэтому вызывается на каждом шаге и каждый раз даёт новое число.

Живой кодинг: как часто выпадает 7?

Бросаем два кубика 10 раз и считаем, сколько раз сумма равна 7.

1. import random.
2. Счётчик sevens = 0 (занятие 1).
3. Цикл на 10 бросков: два randint, их сумма.
4. Печатаем каждый бросок.
5. Если сумма 7 — sevens += 1 (тема 1.4).
6. После цикла — итог.

Вывод в консоли (один запуск)

```
Бросок 1: 3 + 4 = 7
Бросок 2: 2 + 6 = 8
Бросок 3: 6 + 5 = 11
Бросок 4: 4 + 4 = 8
Бросок 5: 2 + 6 = 8
Бросок 6: 3 + 6 = 9
Бросок 7: 4 + 6 = 10
Бросок 8: 2 + 3 = 5
Бросок 9: 1 + 1 = 2
Бросок 10: 5 + 6 = 11
Семёрка выпала 1 раз
```

Вложенный цикл + random: генератор уровня

main.py

```
import random

# Генератор уровня: . — пусто, * — бонус
for row in range(4):
    for col in range(12):
        print(random.choice("....*"), end="")
    print()
```

Вывод в консоли

```
.***....*...
*.**.....*..
....*...**..
..**.*......
```

В строке "....*" четыре точки и одна звёздочка: пустых клеток выпадает больше.

Правило: random.randint(a, b) и random.choice(...) дают новый результат при каждом вызове, а у randint обе границы входят.

Проверь себя

1. Какие числа может выдать random.randint(0, 3)? А какие даст range(0, 3)?

2. Что случится, если убрать строку import random? Как это понять по traceback?

3. Как сделать, чтобы в генераторе уровня бонусов-звёздочек стало больше?

4. Почему в живом кодинге sevens = 0 стоит перед циклом, а не внутри него?

Задание: «Генератор паролей»

Утилита, которой можно пользоваться: надёжный пароль для новой игры или соцсети.

- Программа спрашивает: «Длина пароля: ».
- Выводит «Твой пароль: » и пароль ровно такой длины из случайных букв и цифр.
- При каждом запуске пароль новый.

Подсказка. Запиши все разрешённые символы в константу SYMBOLS. Похожие 0 и O, 1 и l в неё не клади — пароль будет легче переписать. Начни с пустой строки password = "" и в цикле добавляй по символу через +=.

★ **Успел раньше?** Спроси ещё «Сколько паролей?» и выведи столько вариантов на выбор, каждый с номером.

Что должно получиться

Запуск 1

```
Длина пароля: 12   <- ввёл пользователь
Твой пароль: RsMVEXjsnqzh
```

Запуск 2

```
Длина пароля: 8   <- ввёл пользователь
Твой пароль: e6VwVfkk
```

Стрелка — пометка для тебя, в программе её нет. Пароли у тебя будут другие.

Проверь себя

- Посчитай символы: их ровно столько, сколько ввёл.
- Запусти ещё раз — пароль другой.
- Пароль выводится один раз, а не по кусочку.

★★ «Одна звезда на небе»

Нарисуй ночное небо: поле 5 строк × 20 столбцов из точек. Ровно в одной случайной клетке горит звезда *.

Генератор уровня с прошлого слайда не подойдёт: там звёзд может быть сколько угодно, и даже ни одной. Придумай, как гарантировать ровно одну.

Запусти 3 раза: звезда каждый раз в новом месте, но всегда одна.



Урок 3 • 22 минуты

Блок 3 из 3. else у цикла и while True

В этом блоке ты научишься узнавать, что цикл прошёл до конца без break, и писать программу, которая работает, пока её не остановят.

else у цикла: «если не нашли»

else после цикла выполняется, если цикл закончился сам, а не через break.

Как охранник обходит все двери. Нашёл открытую — сразу поднимает тревогу (break). Обошёл все, а открытой нет — пишет в журнал «всё в порядке» (else).

Зачем. Без else пришлось бы заводить лишнюю переменную-пометку «нашли или нет».

Шаблон

```
for ... in ...:
    if нашли:
        break
else:  # на уровне for!
    # только если break
    # не случился
    print("не нашли")
```

Пример: можно ли взять такой ник?

main.py

```
nickname = input("Ник: ")
for symbol in nickname:
    if symbol == " ":
        print("Пробел в нике нельзя")
        break
else:
    print("Ник подходит")
```

Запуск 1

```
Ник: Shadow_Fox <- ввёл
Ник подходит
```

Запуск 2

```
Ник: Shadow Fox <- ввёл
Пробел в нике нельзя
```

Разбор

Цикл проверяет символы ника по очереди.

Запуск 1: пробела нет, цикл дошёл до конца — сработал else.

Запуск 2: нашёлся пробел, break прервал цикл — else пропущен.

Стрелка «<- ввёл» — пометка: это ввёл пользователь, в программе её нет.

Ловушка: else не под тем словом

Сломано: else под if

main.py

```
nickname = input("Ник: ")
for symbol in nickname:
    if symbol == " ":
        print("Пробел в нике нельзя")
        break
    else:
        print("Ник подходит")
```

Вывод в консоли

```
Ник: Fox 7   <- ввёл пользователь
Ник подходит
Ник подходит
Ник подходит
Пробел в нике нельзя
```

Правильно: else на уровне for

main.py

```
nickname = input("Ник: ")
for symbol in nickname:
    if symbol == " ":
        print("Пробел в нике нельзя")
        break
else:
    print("Ник подходит")
```

Вывод в консоли

```
Ник: Fox 7   <- ввёл пользователь
Пробел в нике нельзя
```

while True: пока не скажут «хватит»

while True — цикл, условие которого всегда верно. Сам он не закончится, выйти можно только через break. Удобно, когда заранее неизвестно, сколько раз повторять.

main.py

```
while True:
    command = input("Команда (стоп — выход): ")
    if command == "стоп":
        break
    print(f"Выполняю: {command}")
print("Программа завершена")
```

Вывод в консоли

```
Команда (стоп — выход): прыжок <-
Выполняю: прыжок
Команда (стоп — выход): бег <-
Выполняю: бег
Команда (стоп — выход): стоп <-
Программа завершена
```

Зависла программа? Ctrl+C в терминале или кнопка «Стоп» в Thonny. Стрелка <- — пометка: это ввёл пользователь, в программе её нет.

Пример: разблокировка телефона

main.py

```
SECRET_PIN = "4821"
attempts = 0
while True:
    pin = input("PIN: ")
    attempts += 1
    if pin == SECRET_PIN:
        break
    print("Неверно")
print(f"Телефон разблокирован. Попыток: {attempts}")
```

Вывод в консоли

```
PIN: 1111   <- ввёл пользователь
Неверно
PIN: 4812   <- ввёл пользователь
Неверно
PIN: 4821   <- ввёл пользователь
Телефон разблокирован. Попыток: 3
```

Стрелка — пометка для тебя, в программе её нет.

Сколько попыток понадобится, заранее неизвестно, поэтому while True. Счётчик attempts — как в занятии 1.

Когда что выбирать

Запись	Когда нужна	Главное
for … else	проверить все варианты и узнать, что ничего не нашли	else — только если не было break
while True	повторять, пока пользователь не скажет «хватит»	внутри обязательно break

Правило: else у цикла стоит на уровне for и срабатывает, только если цикл не прервал break; while True без break не закончится никогда.

Проверь себя

1. Что выведет пример с ником, если сразу нажать Enter и ввести пустой ник?

2. Что будет, если в программе с командами убрать break?

3. Почему в ловушке «Ник подходит» напечаталось три раза?

Задание: «Сейф с PIN-кодом»

У игрока 3 попытки, чтобы открыть сейф в квесте.

- Программа спрашивает: «Попытка 1. Код: », потом «Попытка 2. Код: » и так далее.
- Верный код — пишет «Сейф открыт» и больше не спрашивает.
- Неверный — пишет «Неверный код».
- Если все 3 попытки неверные — «Сейф заблокирован».

Подсказка. Попыток ровно три — значит for, а не while True. Что написать, если не угадал ни разу? Вспомни, когда срабатывает else у цикла.

★ **Успел раньше?** После неверного кода подсказывай: «Код больше» или «Код меньше».

Что должно получиться

```
Запуск 1: угадал со второй попытки
Попытка 1. Код: 1234   <- ввёл
Неверный код
Попытка 2. Код: 7395   <- ввёл
Сейф открыт
```

```
Запуск 2: три ошибки
Попытка 1. Код: 0000   <- ввёл
Неверный код
Попытка 2. Код: 1111   <- ввёл
Неверный код
Попытка 3. Код: 2222   <- ввёл
Неверный код
Сейф заблокирован
```

Проверь себя

- После верного кода программа больше не спрашивает и не пишет «Сейф заблокирован».
- Четвёртой попытки нет.
- Стрелка «<- ввёл» — пометка для тебя, в программе её нет. Код сейфа в примере — 7395.

★★ «Простое или составное?»

Программа спрашивает число и говорит, простое оно или составное. Если составное — называет самый маленький делитель.

Простое число делится без остатка только на 1 и на себя. Придумай, какие делители нужно перебрать и как использовать то, что «не нашли ни одного».

```
Запуск 1
Число: 97   <- ввёл
97 — простое
```

```
Запуск 2
Число: 91   <- ввёл
91 — составное, делится на 7
```

Финал • Вопросы по всей теме 1.5

Связываем всё вместе

1. Чем while True с break отличается от while с условием из занятия 1? Когда что удобнее?

2. В «Дуэли кубиков» 5 раундов, в каждом 3 броска. Какие нужны циклы и какой внутри какого?

3. Что будет, если в «Сейфе» поменять range(1, 4) на range(1, 3)?

Мини-проект: «Дуэль кубиков»

Ты и компьютер 5 раундов бросаете по 3 кубика. Больше сумма — раунд твой. В конце — счёт и победитель, а потом вопрос: сыграем ещё?

Пример игры от запуска до конца. Числа случайные — у тебя будут другие.

Стрелка — пометка для тебя, в программе её нет.

Вывод в консоли

```
Раунд 1. Ты: 5 1 1 = 7, компьютер: 16
Раунд 2. Ты: 4 4 5 = 13, компьютер: 12
Раунд 3. Ты: 6 6 5 = 17, компьютер: 9
Раунд 4. Ты: 3 5 5 = 13, компьютер: 15
Раунд 5. Ты: 4 5 5 = 14, компьютер: 12
Счёт 3:2
Ты выиграл дуэль
Сыграем ещё? (да/нет): да   <- ввёл пользователь
Раунд 1. Ты: 1 6 2 = 9, компьютер: 9
Раунд 2. Ты: 6 6 2 = 14, компьютер: 11
Раунд 3. Ты: 1 6 4 = 11, компьютер: 17
Раунд 4. Ты: 3 4 6 = 13, компьютер: 13
Раунд 5. Ты: 4 1 2 = 7, компьютер: 6
Счёт 2:1
Ты выиграл дуэль
Сыграем ещё? (да/нет): нет   <- ввёл пользователь
Спасибо за игру
```

Что умеет игра

Бросает за тебя

3 кубика, показывает каждый и сумму.

Бросает за компьютер

3 кубика и показывает его сумму.

Считает счёт

5 раундов, потом счёт и победитель или ничья.

Предлагает реванш

«да» — новая дуэль со счётом 0:0, другой ответ — выход.

Это пригодится в Проекте 1 «Консольные игры»: игра проходится до конца, считает очки и предлагает сыграть ещё раз.

План по шагам

Шаг	Что сделать	Что понадобится	Как проверить
1	Цикл на 3 броска игрока: печатай каждый кубик и копи сумму	вложенный цикл, randint — 1.5	Запусти: «Раунд 1. Ты: 4 4 3» — и так 5 раундов
2	Сумма 3 бросков компьютера, вывод обеих сумм, +1 победа тому, у кого больше	if и elif — 1.4	В конце строки: «= 11, компьютер: 9»
3	После 5 раундов: счёт и победитель или «Ничья»	if, elif, else — 1.4	При счёте 3:2 — «Ты выиграл дуэль»
4	Если ответ не «да» — выйти из while True	break — 1.5	«нет» — «Спасибо за игру»; «да» — новая дуэль 0:0

Проверяй после каждого шага, а не в конце. Застрял — читай traceback снизу вверх, а не меняй код наугад.

Заготовка

duel.py

```
import random

while True:
    player_wins = 0
    computer_wins = 0
    for round_number in range(1, 6):
        print(f"Раунд {round_number}. Ты:", end=" ")
        player_sum = 0
        # Шаг 1: цикл на 3 броска — печатай кубик, копи player_sum
        # Шаг 2: сумма компьютера, вывод обеих сумм, кому победа
    # Шаг 3: счёт и победитель (или «Ничья»)
    again = input("Сыграем ещё? (да/нет): ")
    # Шаг 4: если ответ не «да» — выйти из цикла
print("Спасибо за игру")
```

Заготовка уже запускается. Пока нет шага 4, игра не закончится: выход — Ctrl+C или «Стоп» в Thonny.

Готово, если…

Блок 1. В каждом из 5 раундов видно 3 кубика игрока и обе суммы.

Блок 2. При новом запуске числа другие, каждое от 1 до 6.

Блок 3. На «нет» программа пишет «Спасибо за игру» и завершается; на «да» — новая дуэль со счётом 0:0.

Итог. При равном счёте выводится «Ничья».

Объясни каждую строку своего кода: зачем она и что будет без неё.

★ Бонус-бросок

После раунда у игрока до 3 бросков, чтобы выбить шестёрку. Выбил — +1 победа. Нет — «Бонуса нет».

```
Было: Раунд 1. Ты: …
Станет: + строка «Бонус: 2 5 6 — шестёрка, +1 победа»
```

★★ Статистика сессии

После каждой дуэли: сколько дуэлей сыграно и сколько из них выиграл ты.

```
Станет: «Дуэлей: 2, твоих побед: 2»
```

Челлендж: «Гонка улиток»

Три улитки ползут к финишу на клетке 20. Перед стартом игрок ставит на улитку 1, 2 или 3.

- Каждый ход каждая улитка проползает случайно от 0 до 3 клеток.
- После хода рисуй дорожки: точки — пройденный путь, буква @ — улитка.
- Кто первым дошёл до 20 — победил. Угадал ли игрок?
- Ходы идут, пока кто-то не финишировал, — заранее неизвестно, сколько их.

Пример кадра гонки

```
1 ........@
2 .....@
3 ..........@
```

Образец того, как могут выглядеть дорожки. Свой формат — можно.

Всё нужное ты уже знаешь: случайные числа, циклы в цикле, бесконечный цикл с выходом.

Итоги темы 1.5 «Циклы»

Теперь я умею

- повторять действия, пока условие верно, и считать счётчиком и накопителем;
- перебирать числа и символы строки циклом for и range;
- выходить из цикла и пропускать шаг: break и continue;
- рисовать таблицы, поля и фигуры из символов циклом в цикле;
- получать случайные числа и символы: randint и choice;
- узнавать, что цикл прошёл до конца без break;
- писать программу, которая работает, пока её не остановят, и останавливать зависшую.

Дальше: тема 1.6 «Отладка» и Проект 1 «Консольные игры».